

Accounting Web App Project Flow & System Design

Next.js + Node.js / Express.js + MongoDB

Sales · Expenditure · Customer Logbook · Dashboard

Document Contents

1. Project Overview
2. Goals & Core Principles
3. Tech Stack & Architecture
4. Database Design (MongoDB Collections)
5. Menu / Price Master & Auto-Calculation Logic
6. Backend API Design (Express Routes)
7. Frontend Page & Component Flow (Next.js)
8. Key User Flows
9. Filtering & Search Logic
10. Dashboard Logic (Monthly & Overall)
11. Folder Structure (Frontend + Backend)
12. Validation, Edge Cases & Business Rules
13. Suggested Build Roadmap (Phases)
14. Future Enhancements (Optional)

1. Project Overview

Restro Rasoi is a home-run cloud kitchen. This web application will replace manual registers/notebooks with a simple, reliable digital system to record every sale and every expense, track customers, and instantly see whether the business is profitable — both for the current month and since the kitchen started.

The system is built around four core modules that mirror how the business actually works day-to-day:

- **Sales** — every order placed, who ordered it, what was ordered, how much it cost, and whether/how it was paid.
- **Expenditure** — every rupee spent on ingredients, packaging, gas, delivery, etc.
- **Customer Logbook** — a running address book of everyone who has ordered, built automatically from sales entries.
- **Dashboard** — a real-time summary of sales, expenditure and net profit, viewable monthly or overall.

A core convenience feature is the **Menu Master**: a pre-fed list of dishes and their prices. When recording a sale, selecting items from this list auto-calculates the order amount, so only the delivery charge needs to be entered manually. Items not in the master list can be added on the fly via an **"Other"** option.

2. Goals & Core Principles

- **Speed of data entry** — Mom should be able to log a sale or expense in under 30 seconds on a phone screen.
- **Zero manual math** — prices, totals and profit are always calculated by the system, never typed in by hand (except delivery charge and 'Other' item price).
- **Single source of truth** — all data lives in MongoDB; the dashboard is simply a live aggregation of Sales and Expenditure, never a separately maintained number.
- **Mobile-first, simple UI** — large touch targets, minimal typing, dropdowns over free text wherever possible.
- **Searchable history** — every table (Sales, Expenditure, Customers) must be filterable so past records are easy to find for reconciliation or disputes.

3. Tech Stack & Architecture

Layer	Choice & Role
Frontend	Next.js (React) — pages/app router, server-side rendering for the dashboard, client-side forms for fast data entry.
Backend	Node.js + Express.js — REST API exposing CRUD endpoints for Sales, Expenditure, Customers and Menu items.
Database	MongoDB (via Mongoose ODM) — flexible document store, good fit for order line-items of variable length.
Auth (optional)	Simple single-user/family login (JWT or session cookie) since this is a private family business tool, not multi-tenant.
Hosting	Frontend on Vercel; Backend on Render/Railway; Database on MongoDB Atlas (free tier is sufficient at this scale).

Layer	Choice & Role
State/Data fetching	React Query / SWR on the frontend for caching API responses and auto-refreshing dashboard numbers.

High-level architecture flow:

Browser (Next.js pages) → calls REST API (Express routes) → Controllers run business logic (price lookup, totals) → Mongoose models read/write MongoDB → JSON response → Next.js renders tables, forms and dashboard charts.

4. Database Design (MongoDB Collections)

Five collections form the backbone of the system. Sales references Customer (by name/phone) and Menu (for item pricing), but stores a snapshot of price/name at time of sale so historic records never change even if menu prices are updated later.

4.1 Sale

Field	Type	Notes
date	Date	Date of the order (defaults to today).
customerName	String	Free text, but auto-suggested from Customer Logbook.
customerPhone	String	Used to link/auto-create a Customer Logbook entry.
items	Array of SaleItem	Each: {itemName, quantity, unitPrice, lineTotal, isCustom}. isCustom=true for 'Other' items typed manually.
deliveryCharge	Number	Only manual numeric input required for a sale.
itemsTotal	Number	Auto-computed = sum of all lineTotal in items.
grandTotal	Number	Auto-computed = itemsTotal + deliveryCharge.
paymentStatus	Enum	'Paid' 'Unpaid'.
paymentMode	Enum	'Cash' 'UPI' 'Card' 'Bank Transfer' 'Other' (disabled/optional when status = Unpaid).
notes	String (optional)	Any special remark on the order.
createdAt / updatedAt	Timestamp	Auto-managed by Mongoose.

4.2 Expenditure

Field	Type	Notes
date	Date	Date the expense was incurred.
category	Enum/String	e.g. Ingredients, Packaging, Gas, Delivery, Utilities, Maintenance, Other — helps with future reporting.
items	String	Description of what was purchased.

Field	Type	Notes
amount	Number	Manually entered expense amount.
notes	String (optional)	Optional remark / vendor name.
createdAt / updatedAt	Timestamp	Auto-managed.

4.3 Customer

Field	Type	Notes
name	String	Required.
phone	String	Unique key used to detect repeat customers.
address	String	Delivery address.
totalOrders	Number	Auto-incremented counter (derived, can be computed on read instead of stored).
totalSpend	Number	Sum of grandTotal across that customer's sales (derived/aggregated, not separately maintained).
createdAt / updatedAt	Timestamp	Auto-managed.

4.4 MenuItem (Price Master)

Field	Type	Notes
name	String	Dish name, e.g. 'Chicken Biryani (Full)'.
category	String (optional)	e.g. Starters, Main Course, Beverages — for grouping in the dropdown.
price	Number	Standard selling price, auto-filled into a sale's line item.
isActive	Boolean	Soft-disable an item without deleting sale history.
createdAt / updatedAt	Timestamp	Auto-managed.

Why snapshot pricing in Sale.items instead of always looking up MenuItem live? If the menu price of Chicken Biryani changes next month, last month's sales must still reflect the price actually charged at that time. So each SaleItem stores its own unitPrice copied from MenuItem at the moment of entry.

4.5 User (optional, for login)

Field	Type	Notes
name	String	e.g. 'Mom', 'Admin'.
email/username	String	Login identifier.
passwordHash	String	Bcrypt-hashed password.
role	Enum	'owner' 'staff' (future-proofing if helpers are added later).

5. Menu / Price Master & Auto-Calculation Logic

This is the feature that removes manual math from daily sales entry. The flow when adding a new Sale works as follows:

- The Menu Master is seeded once (via an admin 'Manage Menu' screen or a seed script) with every dish and its current price.
- On the 'New Sale' form, the item-selection field is a searchable multi-select dropdown populated from *GET /api/menu*.
- Selecting a dish auto-fills its unitPrice; the user only sets the quantity. $\text{lineTotal} = \text{unitPrice} \times \text{quantity}$ is computed instantly in the UI.
- Multiple dishes can be added to one order, each becoming a row in the items array.
- If the dish isn't in the list, the user picks **'Other'** at the bottom of the dropdown, which reveals two plain text/number inputs: item name and price. This row is flagged `isCustom = true`.
- `itemsTotal` auto-sums every line. The user then types only the **delivery charge** (a plain number field) — this is the only price-related manual entry in the whole form.
- `grandTotal = itemsTotal + deliveryCharge` is shown live before saving.
- Optionally, a 'Save as new menu item' checkbox next to a custom item lets the owner promote a one-off 'Other' dish into the permanent Menu Master for next time.

Manage Menu screen (admin-only, simple CRUD):

- Add new dish (name, category, price).
- Edit price of an existing dish (does not retroactively change past sales — see 4.1 note on price snapshotting).
- Mark a dish inactive/active (e.g. seasonal items) instead of deleting it, so historical sales referencing it stay intact.

6. Backend API Design (Express Routes)

6.1 Sales

Method	Endpoint	Description
GET	/api/sales	List sales with query-string filters: date range, customerName, item, paymentStatus, paymentMode, page, limit.
GET	/api/sales/:id	Get a single sale's full detail.
POST	/api/sales	Create a sale. Server recomputes <code>itemsTotal/grandTotal</code> from <code>items + deliveryCharge</code> for integrity (never trusts client totals blindly). Auto-creates/updates the Customer record.
PUT	/api/sales/:id	Edit a sale (e.g. mark Unpaid → Paid, fix a typo).
DELETE	/api/sales/:id	Remove a wrongly entered sale.

6.2 Expenditure

Method	Endpoint	Description
GET	/api/expense	List with filters: date range, category, min/max amount.
GET	/api/expense/:id	Get a single expense record.
POST	/api/expense	Create a new expense entry.
PUT	/api/expense/:id	Edit an expense.
DELETE	/api/expense/:id	Remove an expense.

6.3 Customer Logbook

Method	Endpoint	Description
GET	/api/customers	List customers, with search by name/phone, sortable by totalSpend or totalOrders.
GET	/api/customers/:id	Customer profile + their full order history (joined from Sales).
POST	/api/customers	Manually add a customer (rarely needed — usually auto-created from a Sale).
PUT	/api/customers/:id	Edit address/phone.
DELETE	/api/customers/:id	Remove a customer record.

6.4 Menu Master

Method	Endpoint	Description
GET	/api/menu	List all active menu items (used to populate the Sale form dropdown).
POST	/api/menu	Add a new dish.
PUT	/api/menu/:id	Update name/price/category or toggle active status.
DELETE	/api/menu/:id	Permanently remove a dish (only if never used in a sale; otherwise prefer deactivating).

6.5 Dashboard / Reports

Method	Endpoint	Description
GET	/api/dashboard/summary	Returns overall totals: totalSales, totalExpenditure, netProfit, totalOrders, unpaidAmount.
GET	/api/dashboard/monthly?month=&year;=	Same breakdown scoped to one calendar month.
GET	/api/dashboard/trend?range=12m	Month-by-month series of sales/expenditure/profit, for the dashboard chart.

All list endpoints (Sales, Expenditure, Customers) support pagination (page, limit) and sorting, so the tables stay fast even after years of data.

7. Frontend Page & Component Flow (Next.js)

Route	Purpose
/	Dashboard (default landing page after login) — summary cards + chart.
/sales	Sales table with filters + 'Add Sale' button opening a modal/drawer form.
/sales/new	(or modal) Add Sale form with menu-driven item picker.
/sales/[id]	View/edit a single sale.
/expenditure	Expenditure table with filters + 'Add Expense' button.
/expenditure/new	(or modal) Add Expense form.
/customers	Customer Logbook table, searchable.
/customers/[id]	Customer profile: contact details + their order history.
/menu	Admin screen to manage the Menu Master (CRUD on dishes/prices).
/login	Simple auth screen (if login is included).

Shared/reusable components:

- **AppShell** — top navbar / sidebar with links to Dashboard, Sales, Expenditure, Customers, Menu.
- **FilterBar** — generic, reusable filter row (date range picker, dropdowns, text search) that emits a query object consumed by table fetch hooks.
- **DataTable** — reusable paginated/sortable table component used by Sales, Expenditure and Customers pages.
- **ItemPicker** — searchable dropdown of MenuItem's with quantity stepper and the built-in 'Other' option; used inside the Sale form.
- **SummaryCard** — single stat tile (Total Sales / Total Expenditure / Net Profit / Unpaid Amount) used on the Dashboard.
- **TrendChart** — line/bar chart (e.g. via Recharts) plotting monthly sales vs expenditure vs profit.

8. Key User Flows

8.1 Recording a Sale

- Open /sales → click 'Add Sale'.
- Date defaults to today (editable).
- Type customer name — if an existing customer matches by name/phone, autocomplete fills phone & address; otherwise these are typed fresh.
- Add one or more items via the ItemPicker (menu items auto-price; 'Other' allows manual name + price).
- Enter delivery charge.
- Grand total displays live.
- Toggle Paid/Unpaid; if Paid, select payment mode (Cash/UPI/Card/etc.).
- Save → POST /api/sales → backend recalculates totals, upserts the Customer record, and the new row appears at the top of the Sales table.

8.2 Recording an Expense

- Open /expenditure → click 'Add Expense'.
- Enter date, choose/enter category, describe item(s), enter amount.
- Save → POST /api/expenditure → row appears in the table and feeds into dashboard totals immediately.

8.3 Looking up a Customer

- Open /customers → search by name or phone.
- Click a customer → see address, phone, and a full timeline of every order they've placed with amounts and paid/unpaid status (handy for following up on dues).

8.4 Checking Business Health

- Open / (Dashboard) → see four summary cards: Total Sales, Total Expenditure, Net Profit, and Unpaid Dues — toggle between 'This Month' and 'All Time'.
- Scroll to the trend chart to see how the last several months compare.

9. Filtering & Search Logic

9.1 Sales table filters

- Date range (from – to), with quick presets: Today, This Week, This Month, This Year.
- Customer name (text search, partial match).
- Item ordered (dropdown of menu items, matches any sale containing that item).
- Payment status: Paid / Unpaid / All.
- Payment mode: Cash / UPI / Card / Bank Transfer / Other / All.
- Combined filters are AND-ed together and sent as query parameters to GET /api/sales.

9.2 Expenditure table filters

- Date range, with the same quick presets.
- Category (Ingredients / Packaging / Gas / Delivery / Utilities / Other / All).
- Amount range (min – max).

9.3 Customer table filters

- Search by name or phone number.
- Sort by total orders or total spend, to surface top customers.

Implementation note: filters are translated into MongoDB query conditions in the controller (e.g. `$gte/$lte` on date and amount, `$regex` for name search, exact match for status/mode/category), keeping all filtering logic server-side so it scales as data grows.

10. Dashboard Logic (Monthly & Overall)

The dashboard never stores its own numbers — every figure is computed on demand from Sales and Expenditure using MongoDB aggregation pipelines, so it is always accurate and never goes out of sync.

Metric	Calculation
Total Sales	Sum of <code>grandTotal</code> across all Sale documents in the selected period (month or all-time).
Total Expenditure	Sum of <code>amount</code> across all Expenditure documents in the selected period.
Net Profit	Total Sales – Total Expenditure.
Unpaid Dues	Sum of <code>grandTotal</code> where <code>paymentStatus</code> = 'Unpaid', shown separately so cash-flow gaps are visible even though it's technically counted inside Total Sales.
Monthly Trend	Aggregation grouped by <code>year+month</code> over the last N months, returning {month, sales, expenditure, profit} for charting.
Top Items (optional)	Aggregation unwinding <code>Sale.items</code> , grouped by <code>itemName</code> , summed by <code>quantity/lineTotal</code> — useful for spotting best-sellers.

The Dashboard page offers a simple toggle: **This Month** vs **All Time**, which switches the API call between `/api/dashboard/monthly` and `/api/dashboard/summary`.

11. Folder Structure (Frontend + Backend)

11.1 Frontend (Next.js)

```
restro-rasoi-frontend/
|-- app/                                (or pages/, depending on router choice)
|   |-- page.jsx                        -> Dashboard
|   |-- sales/
|       |-- page.jsx                    -> Sales table + filters
|       |-- [id]/page.jsx                -> Sale detail/edit
|   |-- expenditure/
|       |-- page.jsx                    -> Expenditure table + filters
|   |-- customers/
|       |-- page.jsx                    -> Customer logbook
|       |-- [id]/page.jsx                -> Customer profile
|   |-- menu/page.jsx                    -> Menu master CRUD
|-- components/
|   |-- AppShell.jsx
|   |-- DataTable.jsx
|   |-- FilterBar.jsx
|   |-- ItemPicker.jsx
|   |-- SummaryCard.jsx
|   |-- TrendChart.jsx
|   |-- forms/
|       |-- SaleForm.jsx
|       |-- ExpenseForm.jsx
|-- lib/
|   |-- apiClient.js                    -> fetch wrapper / axios instance
|   |-- hooks/                          -> useSales, useExpenditure, useCustomers, useDashboard
|-- styles/
|-- public/
```

11.2 Backend (Node.js + Express)

```
restro-rasoi-backend/
|-- src/
|   |-- models/
|       |-- Sale.js
|       |-- Expenditure.js
|       |-- Customer.js
|       |-- MenuItem.js
|       |-- User.js
|   |-- routes/
|       |-- sales.routes.js
|       |-- expenditure.routes.js
|       |-- customers.routes.js
|       |-- menu.routes.js
|       |-- dashboard.routes.js
|   |-- controllers/
|       |-- sales.controller.js
|       |-- expenditure.controller.js
|       |-- customers.controller.js
|       |-- menu.controller.js
|       |-- dashboard.controller.js
|   |-- middleware/
|       |-- auth.js
|       |-- errorHandler.js
|   |-- config/
|       |-- db.js                        -> MongoDB connection
|       |-- app.js
|-- .env                                -> MONGO_URI, JWT_SECRET, PORT
```

```
`-- server.js
```

12. Validation, Edge Cases & Business Rules

- A Sale must have at least one item before it can be saved.
- If `paymentStatus = 'Unpaid'`, `paymentMode` is not required (can be filled later when marked Paid).
- Phone number format should be loosely validated (10 digits) but not block saving if a customer only gives a name.
- Editing a Sale's items must re-run the same total-calculation logic used on create, so totals can never drift from the line items.
- Deleting a Sale should NOT delete the associated Customer (a customer may have other orders); customer totals should simply be recalculated.
- Deactivating (not deleting) a MenuItem keeps historical Sales intact even if a dish is removed from the current menu.
- All money fields are stored in a single currency (INR) as numbers (paise-level precision optional); never store currency as a formatted string.
- Dates should be stored in UTC and displayed in local (IST) format for consistent monthly grouping regardless of where the dashboard is viewed from.

13. Suggested Build Roadmap (Phases)

Phase	Deliverables
Phase 1 — Foundation	Set up Next.js + Express + MongoDB Atlas. Build MenuItem model/CRUD and seed the initial menu with real dishes/prices.
Phase 2 — Sales	Build Sale model, API, the Add Sale form with ItemPicker (incl. 'Other'), and the Sales table with filters.
Phase 3 — Expenditure	Build Expenditure model, API, Add Expense form, and filtered table.
Phase 4 — Customers	Build Customer model + auto-creation hook from Sales, Customer Logbook table and profile page.
Phase 5 — Dashboard	Build aggregation endpoints and the Dashboard UI (summary cards + monthly/overall toggle + trend chart).
Phase 6 — Polish	Auth/login, mobile responsiveness pass, empty states, loading skeletons, basic error handling, deploy to Vercel + Render + Atlas.

14. Future Enhancements (Optional)

- Export Sales/Expenditure to Excel/PDF for a chosen date range.
- WhatsApp/SMS reminder for customers with unpaid dues.
- Top-selling items report and simple inventory/stock tracking tied to ingredients.
- Multi-user roles if helpers/staff start taking orders.
- Daily auto-summary notification (e.g. end-of-day sales total via WhatsApp).

Next step: once this flow is confirmed, frontend design instructions (colors, layout, component styling) can be applied on top of this structure to start building the UI.

15. Role-Based Access & Accessibility (Addendum)

This section refines the optional Auth module (see 3 and 4.5) into two concrete roles, and adds an explicit usability mandate so the app stays approachable for every family member who touches it.

15.1 Two-Role Access Model

The User collection's role enum is fixed to two practical roles instead of a generic owner/staff split:

Role	Can Access	Typical User
Owner	Add Sale, Add Expense (day-to-day data entry only)	Mom — daily operator
Admin	Everything Owner can do, plus: Manage Menu, view/edit Customers, edit or delete past Sales/Expenditure, view Dashboard, manage users	Family member handling oversight

Enforcement notes:

- Route-level: /sales/new and /expenditure/new are reachable by both roles; /menu, /customers, edit/delete actions on existing Sales or Expenditure, and the Dashboard are gated to Admin only.
- Backend-level: every protected Express route checks req.user.role via the existing auth.js middleware (e.g. requireRole('admin')) — the frontend hiding a button is a convenience, not the actual security boundary.
- Login stays simple: a single shared login screen (/login) that routes to the same AppShell, just with Admin-only nav links hidden for an Owner session.
- Dashboard is intentionally Admin-only at this stage, since it surfaces full business profitability — this can be revisited later if Mom should see it too.

15.2 Universal Usability ('Anyone, Any Age' Principle)

This extends Section 2's existing 'Mobile-first, simple UI' goal into a firmer design rule: the Sale and Expense forms — the screens used daily — must be operable by someone with no technical background and no training.

- Large tap targets (minimum ~44px) and big, legible fonts throughout — no dense, small-text tables on the entry screens.
- Icons paired with plain-language labels, never icon-only buttons (e.g. a trash icon always sits next to the word 'Delete').
- Maximum use of dropdowns, steppers and toggles over free typing — already the system's principle for menu items, extended consistently to payment status, payment mode and category fields.
- Every destructive action (delete sale, delete expense) requires a simple one-tap confirmation dialog in plain language: 'Remove this order?' rather than a technical warning.
- High-contrast colour palette and a clear visual difference between Paid/Unpaid and Sales/Expenditure, so status is readable at a glance.
- Errors are shown as short, friendly inline messages near the relevant field (e.g. 'Please add at least one item') rather than technical alert popups or console-style errors.

- The login screen itself stays minimal — just a name/role picker and password — so it doesn't become a barrier before the actual work begins.

Net effect: the role split keeps daily entry frictionless for Mom while protecting menu pricing, customer data and historical records behind Admin oversight — and the accessibility rules ensure neither role ever needs a tutorial to use the app.

15.3 Persistent Sessions (Stay Logged In)

Logging in fresh every time someone wants to log a sale defeats the 'under 30 seconds' goal from Section 2. Both roles get a long-lived session instead of a short one, so the app behaves like a normal phone app that stays signed in until someone deliberately logs out.

- JWT-based session with a long expiry (e.g. 30 days) stored in an HttpOnly cookie, refreshed automatically on each visit so active use keeps extending it — the user is never abruptly logged out mid-week.
- Each role (Owner, Admin) gets its own persisted session, so a shared family device can have both logged in at once without one login overwriting the other (e.g. two saved sessions, switched via a simple role picker).
- auth.js middleware verifies the token silently on every request; if valid, the app opens directly to the Dashboard or Sales screen — no login screen shown at all on repeat visits.
- A visible 'Log out' option remains available (e.g. in AppShell) for the rare case someone wants to switch roles or sign out of a shared device manually.
- If the token does expire or is invalidated, the user is sent back to the same minimal /login screen from 15.2 — login is rare, not routine.